

Actividad 5 | Ae3 | Calificada

Ae3 – Incremento 1 del proyecto

Integración de diseño orientado a objetos y patrones
en el Sistema de Gestión de Tutorías

Estudiante: Xavier Romero Villagómez

Semana: 4

Ponderación: 20 % de la Evaluación I

Fecha límite: 02 de septiembre de 2026

Repositorio GitHub: https://github.com/edisonromero-rgb/Sistemas_Tutorias

2. Estado inicial

Al comenzar este incremento, el repositorio tenía dos árboles de código desconectados:

- `edu.uees.tutorias` (Ae1): el dominio real — Usuario, Estudiante, Docente, HorarioDisponible, Reserva, EstadoReserva, ServicioReservas, un Notificador simple (solo NotificadorCorreo) y ReservaRepository/ReservaRepositoryMemoria.
- `edu.uees.patrones` (Ae2): una demo aislada de Factory Method (Notificador basado en String destinatario) y Builder (un Reserva de juguete con String estudiante/docente, sin relación con el dominio de Ae1).

ServicioReservas construía la reserva con `new Reserva(id, estudiante, docente, horario)` directamente y, tras cada operación, llamaba a mano a `Notificador.notificar(...)`. Antes de modificar nada se corrió `mvn clean test` sobre la línea base: compilaba y las 7 pruebas de `ServicioReservasTest` pasaban.

3. Problemas de diseño identificados

Problema 1 — Reglas de cancelación variables por tipo de reserva

El sistema no distinguía tipos de reserva ni antelación mínima para cancelar. Una tutoría grupal (afecta a varios estudiantes) debería exigir más antelación que una prioritaria. Sin un punto de extensión, esa regla habría terminado como un `if/else` dentro de `ServicioReservas` o de `Reserva`, mezclando una política de negocio variable con la coordinación del caso de uso.

Problema 2 — `ServicioReservas` acoplado a la notificación

Cada operación (crear, confirmar, cancelar, reprogramar, finalizar) terminaba con llamadas manuales a `notificador.notificar(...)`, repitiendo la decisión de quién debe enterarse de cada cambio. Agregar un interesado nuevo (calendario, panel administrativo, auditoría) obligaba a modificar `ServicioReservas` cada vez, violando OCP y mezclando coordinación con notificación (SRP).

4. Patrones seleccionados

4.1 Revisión de Factory Method y Builder (Ae2)

Patrón	Problema que resuelve en mi proyecto	¿Se mantiene?	Justificación
Factory Method	Elegir la implementación de Notificador (correo, SMS, push, WhatsApp) según el canal preferido de una reserva.	Sí	Se reintegra sobre el dominio real; ahora <code>ObservadorNotificaciones</code> lo usa en producción vía <code>NotificadorFactoryProvider</code> . En Ae2 era una demo que nadie invocaba.
Builder	Construir Reserva con datos obligatorios y opcionales sin constructor telescópico.	Sí	Se reintegra sobre el Reserva real (antes construía un objeto de juguete). Ahora es la única forma de crear una Reserva; se le agregó tipo (Strategy) y <code>observador(...)</code> (Observer).

4.2 Patrón nuevo 1 — Strategy (políticas de cancelación)

Elemento	Detalle
----------	---------

Problema real	La antelación mínima para cancelar sin penalización varía según el tipo de reserva, y se espera que la política cambie con el tiempo.
Contexto	<code>ServicioReservas.cancelarReserva(id)</code>
Qué cambia	La regla de antelación mínima por tipo (24h normal, 2h prioritaria, 48h grupal).
Qué permanece estable	El contrato <code>PoliticaCancelacion.validar(Reserva)</code> y el flujo de <code>ServicioReservas</code> .
Patrón / clases	<code>PoliticaCancelacion</code> (interfaz), <code>CancelacionNormalPolitica</code> , <code>CancelacionPrioritariaPolitica</code> , <code>CancelacionGrupalPolitica</code> , resueltas por <code>PoliticaCancelacionProvider</code> según <code>Reserva.getTipo()</code> .
Principio SOLID	OCP (nuevo tipo no toca las políticas existentes) y SRP (la regla no vive en <code>ServicioReservas</code> ni en <code>Reserva</code>).
Beneficio / costo	Cambiar o agregar una política no afecta al resto; una clase por regla nueva (sobreingeniería si solo hubiera un tipo).
Verificación	<code>PoliticaCancelacionTest</code> (4 pruebas) + demo en <code>App.java</code> : una reserva GRUPAL con 18h de antelación es rechazada.

4.3 Patrón nuevo 2 — Observer (cambios de estado de Reserva)

Elemento	Detalle
Problema real	Varios componentes independientes (notificación, calendario, panel administrativo) deben reaccionar cuando cambia el estado de una reserva.
Contexto	<code>Reserva.confirmar()/cancelar()/reprogramar()/finalizar()</code> y su creación.
Qué cambia	La lista de interesados y lo que hace cada uno ante un cambio de estado.
Qué permanece	<code>Reserva</code> no conoce a sus observadores concretos, solo el contrato <code>ReservaObserver</code> .

estable

Patrón / clases	Reserva (Subject), ReservaObserver (contrato), ObservadorNotificaciones, ObservadorCalendario, ObservadorPanelAdministrativo (ConcreteObserver).
Principio SOLID	OCP (nuevo observador no toca Reserva ni ServicioReservas) y SRP (ServicioReservas deja de saber cómo se notifica).
Beneficio / costo	Desacopla el ciclo de vida de sus efectos secundarios; una interfaz y una lista más en Reserva.
Verificación	ServicioReservasTest (observador de prueba) + demo en App.java: correo/SMS, calendario y panel reaccionan sin que ServicioReservas los invoque.

5. SOLID, cohesión y acoplamiento

- **SRP:** Reserva concentra su ciclo de vida y avisa que cambió (no decide cómo); ServicioReservas coordina y ya no notifica ni valida políticas directamente.
- **OCP:** canal nuevo → NotificadorFactory nueva; tipo de reserva nuevo → PoliticaCancelacion nueva; interesado nuevo → ReservaObserver nuevo. Ninguno toca código existente.
- **LSP:** cualquier ReservaObserver, PoliticaCancelacion o NotificadorFactory es intercambiable sin que el cliente lo note.
- **ISP:** ReservaObserver y PoliticaCancelacion tienen un único método cada uno.
- **DIP:** ServicioReservas(ReservaRepository, List<ReservaObserver>) depende de abstracciones, no de implementaciones concretas.
- **Cambio deliberado:** ServicioReservas ya no recibe Notificador por constructor como en Ae1; esa responsabilidad nunca fue suya — con Observer, Reserva la asumió correctamente.

6. UML actualizado

Diagrama completo del incremento 1 (dominio, Builder, Factory Method, Strategy y Observer):

NotificadorFactory / NotificadorFactoryProvider	Creator del Factory Method y el punto que traduce CanalNotificacion a la fábrica concreta correspondiente.
PoliticaCancelacion / PoliticaCancelacionProvider	Strategy de cancelación; el provider resuelve la política según TipoReserva.
ServicioReservas	Coordina repositorio, builder, política de cancelación y observadores por defecto; ya no conoce Notificador directamente.

8. Verificación

mvn clean test — 24 pruebas, 0 fallos:

```
[INFO] Running edu.uees.tutorias.cancelacion.PoliticaCancelacionTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running edu.uees.tutorias.domain.ReservaBuilderTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running edu.uees.tutorias.notification.factory.NotificadorFactoryTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running edu.uees.tutorias.service.ServicioReservasTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0

[INFO] Tests run: 24, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

mvn compile exec:java — flujo completo (Builder + Factory Method + Strategy + Observer):

```
[sms -> Ana Perez] Reserva ...-c055c creada, pendiente de confirmacion.
[sms -> Jaime Sayago] Nueva solicitud de tutoria de Ana Perez.
Estado tras crear: PENDIENTE
[sms -> Ana Perez] Tu reserva ...-c055c fue confirmada.
[calendario] evento creado para ...-c055c (HOR-1 2026-09-08 10:00-11:00)
Estado tras confirmar: CONFIRMADA
Cancelacion rechazada: Politica grupal: la reserva ...-c055c requiere al
menos 48h de antelacion para cancelarse; faltan 18h.
[correo -> ana.perez@uees.edu.ec] Reserva ...097 creada, pendiente de confirmacion.
[correo -> jsayago@uees.edu.ec] Nueva solicitud de tutoria de Ana Perez.
[correo -> ana.perez@uees.edu.ec] Tu reserva ...097 fue cancelada.
```

```
[correo -> jsayago@uees.edu.ec] La reserva ...097 fue cancelada.  
[calendario] evento retirado para ...097  
Estado reserva normal tras cancelar: CANCELADA  
Bitacora del panel administrativo:  
  ...c055c: creada  
  ...c055c: PENDIENTE -> CONFIRMADA  
  ...097: creada  
  ...097: PENDIENTE -> CANCELADA
```

La cancelación rechazada demuestra la política Strategy (reserva GRUPAL con solo 18h de antelación, exige 48h). La bitácora del panel demuestra Observer: un tercer componente se enteró de cada cambio sin que ServicioReservas lo invocara.

9. Git / GitHub

Repositorio: https://github.com/edisonromero-rgb/Sistemas_Tutorias

Historial de commits de este incremento (del más reciente al más antiguo):

Commit	Mensaje
76310a6	docs: actualizar README y decisiones de diseño del incremento 1
e23e9e1	docs: actualizar UML del incremento 1
167ca59	feat: aplicar observer a los cambios de estado de reserva
324d0d7	feat: aplicar strategy a las políticas de cancelacion de reserva
0ac2c8a	feat: integrar Builder y Factory Method al dominio del sistema de tutorias
60596fe	refactor: retirar el prototipo aislado de patrones (Ae2) del proyecto

Los seis commits evidencian evolución progresiva: primero se retira el prototipo aislado de Ae2, luego se reintegran Builder y Factory Method sobre el dominio, después se agrega Strategy de forma independiente, luego Observer (que también conecta Strategy al flujo de cancelación), y finalmente se documenta el incremento (UML y README).

10. Conclusiones

El incremento 1 unificó dos árboles de código que hasta Ae2 vivían desconectados y agregó dos patrones (Strategy y Observer) motivados por problemas concretos, no por completar una lista. El cambio más importante no fue agregar clases sino *quitarle* a ServicioReservas una responsabilidad que nunca debió tener (decidir a quién notificar); Observer permitió que esa decisión la tome cada interesado por su cuenta, y Strategy permitió que la regla de cancelación varíe sin tocar el flujo de coordinación. Factory Method y Builder de Ae2 no se descartaron: se conectaron por primera vez a un flujo real, y aprendí que un patrón que "funciona" en una demo aislada no aporta nada hasta que un caso de uso real depende de él.

11. Uso de inteligencia artificial

Para esta actividad utilicé un asistente de inteligencia artificial (Claude). La herramienta se empleó para integrar el código de Ae2 sobre el dominio de Ae1, generar el código Java de los patrones Strategy y Observer, el diagrama UML y la redacción de este informe y del README, a partir de los requisitos de la guía de Ae3. Revisé, probé (`mvn clean test` y `mvn compile exec:java`, evidencia en la sección 8) y adapté el contenido generado, y puedo explicar y justificar el código y las decisiones de diseño presentadas.